01 / Project snapshot and reading route

IOP - Native development field guide

Updated for offline use on 2026-10-15. Published code baseline: develop e9bd49d (IOP-203). The local Docker installation also contains IOP-211 at b3a4a01 on feature/IOP-211-locator-models-and-access. That review branch is not yet merged or published. This PDF update does not approve its integration or a platform release.

Your selected setup: React/Vite and the NestJS API run on the Mac. PostgreSQL stays in Docker, using the existing iop_local database, platform-data volume, accounts and records. Native refers to the two application processes; Docker Desktop remains necessary for PostgreSQL. Native edits write to the same database.

Read next	What you will learn
02-03 / Module workflows	What Maintenance, Handover views, Workforce, Assets, Tracking and Locator do; who uses them.
04 / Integrations	How external adapters translate source formats into module-owned ports; what is connected and what is still pending.
05-07 / Code map	Repository, backend owners, browser screens and places to implement changes.
08-10 / Start native	Prepare private settings once; reuse the existing Docker database; run compiler, API and Vite.
11-13 / Modify and verify	Follow a change, run checks, handle migrations and return to the existing Docker images.
14-17 / Operate and continue	Local jobs/backup/Audit, remaining inputs, offline routine and canonical sources.

Choose the source deliberately. The commands run the code in your checked-out branch. Published develop includes the new operational modules. To reproduce the currently installed stricter Assets access and latest Locator behavior, use the existing IOP-211 review branch after saving your current work:

```
git status --short --branch
git switch feature/IOP-211-locator-models-and-access
```

The recipe was verified on that IOP-211 source to match this installation. Using older code against its migrated database is not a rollback.

Do not merge that branch merely to follow this guide. Its local PDF descriptions are explicitly marked IOP-211. Ordinary new stories still start from develop under the repository workflow. A code checkout does not roll back database migrations.

Editable source: docs/development/native-development-guide.md. PDF builder: docs/development/pdf/build_native_guide.py. Health proves process liveness only.

02 / Module guide: daily operations

These are cooperating modules in one platform, not separate applications. The current profile and source-specific grants are checked on the server.

Module / screen	What you do and what it means
Start / Administration	See your recorded schedule and assignment, assigned work, Handover highlights and active Information. Administrator also sees account/import summaries. Planned schedules are not attendance; account summaries are not the general Audit feed.
Data Analysis / OIP	Explore historical Executive, Halle, Bereich, Equipment, Error and daily/monthly reports; compare trends, Pareto and KPIs, then inspect contributing source evidence. Administrator imports/prepares files. Alarm frequency and accumulated seconds are source measures, not incident identities or plant downtime. Missing imports remain gaps.
Shift Handover: working views	Meeting preparation (Daily overview for Team Leader), personal Journal and Department matrix show current reports, discussion/highlight flags, pending issues and revisions. Categories include Safety, technical reports, Information, People and Success. Success closes explicitly selected eligible reports/work atomically; it does not close everything in an area.
Shift Handover: formal records	A draft belongs to an actual Workforce shift, location and UTC interval. Review, publish and close the transfer; retain frozen evidence and a separate incoming-manager acknowledgement. Closing a transfer does not resolve current journal issues. Develop has a Formal Handover entry; IOP-211 puts Shift records inside Shift Handover.
Workforce & shifts	Maintain teams, shift definitions, availability schedules, weekly planning, dated assignments, floating duties and revisions. Team Leader plans; Administrator also imports/configures and manages logical removal. Technician/Task Force read operational plans. Availability and operational assignment are separate; there is no payroll or automatic optimization.
Maintenance	Propose and coordinate work, priorities, ownership, assignment, status and outcomes. Review exactly which Handover issues are included/excluded from a repair. Completion resolves eligible included pending issues and retains both histories. Technician accesses assigned work from Start/activity; Team Leader/Administrator coordinate. Ultimo references are manual until its adapter is connected.

Handover viewing is a workflow. There is no separate Handover Viewer module or fifth account profile. Read reports through the named Handover views. Formal records require real location-specific assignments; the system does not invent shifts from alarms or calendar dates. Notifications are in-app, without external email/push.

Administrator, Team Leader and Task Force use formal records and analytical views. All four profiles use ordinary Handover under their grants. Configured Information publication defaults to Team Leader; Administrator's View as does not grant that identity.

03 / Module guide: assets and follow-up

Module / screen	What you do and what remains explicit
Assets / Digital Asset Record	Register a stable equipment identity, maintain exact aliases, validation and retirement, and inspect its source-authorized Handover/Maintenance/analytical timeline. Asset access does not grant access to every source record. Reuse identifiers instead of registering duplicates from image symbols.
Asset management	Define reusable equipment types and typed metadata; record composition parents, controller/channel relations, manufacturer/model/serial and attributed survey reviews. Review source-to-canonical mapping candidates and canonical Asset Pareto. Software review states do not prove a physical inspection. RAW source facts remain unchanged.
Asset Locator	Navigate schematic 3D area scenes, select reusable components, search explicit registered placements and inspect dated problem evidence. Save attributed model revisions when authorized. Separate drawings have no measured common plant origin; coordinates, heights and sensor identity links remain provisional until surveyed. A problem marker is reported evidence, not live health.
Improvement Tracking / Actions	Own an action, hypothesis, goal and dates; link assets/issues, advance its lifecycle, append evidence/corrections and retain server-computed before/after comparisons. Missing coverage and unequal periods remain visible. An observed change does not establish causality. Binary attachment storage is outside this increment.
Area overview and header search	Find authorized areas, assets, reports and work, or combine selected-day journal context with current pending work and exact analytical/formal context. Each source enforces its own access. Current pending work is not reconstructed as historical state at the selected journal date.
Access administration	Administrator manages local accounts, fixed profiles and scoped grants. Password sessions use the retained database. View as previews layout only: it does not change the actor, authority or authorship. Entra contracts are ready; corporate sign-in is not connected.

Published develop versus installed IOP-211: develop permits the three advanced profiles to open Assets/Asset management and edit Locator. IOP-211 restricts those registry/model operations to Administrator. Team Leader and Task Force navigate read-only Locator and operational identity links; Technician rollout remains pending.

IOP-211 also prepares provisional models for all 71 supplied views, identifies motor and photoelectric templates, uses geometry-only navigation with optional editor guides, colors photoelectric components teal and retains camera orientation/zoom on area entry and the full camera on ordinary selection. Published develop has the earlier three traced views and earlier guide/navigation behavior. Neither is a surveyed floorplan.

Fixed profile	Main responsibility
Administrator	Accounts, imports, configuration, advanced operations and IOP-211 registry/model editing.
Team Leader	Planning, operational coordination, Handover and advanced review under current grants.
Task Force	Operational contributions, advanced evidence/action review and authorized work updates.
Technician	Personal Handover, operational plan and own/assigned technical work.

04 / Integrations are external adapters

An integration adapts an external system, component or file format to the contracts that an internal module owns. External field names, protocols and customer-specific mapping belong in the adapter/configuration, not in generic domain rules. The Integrations screen exposes configuration and readiness; it is not proof of a working network connection.

```
External file / provider / identity system
        -> adapter: decode, validate, translate, retain provenance
        -> module-owned port and application use case
        -> scoped domain rules and authorized persistence
host/runtime.ts composes the chosen adapter with the owning module.
```

Existing adapter / contract	Actual readiness
Analytical CSV imports	Working file decoding, RAW/original-file retention, mapping snapshots, source-date admission and atomic publication. A browser upload enters the real import workflow.
Workforce schedule decoding	Working manual-source decoding and preview, followed by explicit validated application to Workforce. A decoded schedule is not an approved assignment.
WinCC aggregates	Read-only provider contract and normalization boundary prepared. Real transport, version/schema, endpoint, credentials and operational verification remain pending. No PLC command or plant control write is included.
Ultimo work orders	Read-only contract prepared for external work references. Manual local text references are not synchronization; actual retrieval/translation and access remain pending.
Entra identity	Verified-identity adapter contract prepared. Token/provider validation, approved tenant/application configuration and real corporate sign-in remain pending. Local password login continues to work.

To implement a real integration: inspect the receiving module's application port, then add a provider transport and source-schema translation in the owning adapter. Validate authentic source data and failures against approved access. Test the port contract, provenance, authorization and idempotency; wire it in host/runtime.ts. Do not bypass business permissions or write directly into another module's tables.

Configuration stores provider/readiness metadata and history. Secret references are not credentials, and configuring a provider does not contact it. A contract-ready state, missing last successful connection and pending handler must stay visible. Real endpoints, schemas, versions and credentials are still required from the plant.

First files: apps/api/src/modules/integrations/application/provider-contracts.ts, adapters/providers/read-only.ts and domain/connections.ts. Working import adapters are under adapters/csv/ and adapters/schedule/. See docs/product/integration-readiness.md.

05 / Architecture and repository layout

IOP is a modular monolith: one API process composes modules with distinct ownership. There are no internal HTTP calls between these modules. Durable local jobs exist, but no continuously deployed worker service is claimed. The separate web and database processes do not turn each business module into a service.

```
Browser: React + Vite       http://127.0.0.1:5173
    | same-origin /api and /health proxy
API: NestJS + module use cases   127.0.0.1:3000
    | pg Pool -> pinned authorized transaction
PostgreSQL in Docker             127.0.0.1:5442
    | existing iop_local database / persistent volume
```

Location	Read or change it for
ARCHITECTURE.md	Accepted implementation boundaries and composition overview.
apps/api/src/main.ts	API bootstrap, shutdown and listener.
apps/api/src/host/	Controllers, DTOs, runtime composition, configuration and HTTP errors.
apps/api/src/modules/	Owning domain rules, application use cases and adapters.
apps/api/src/persistence/site-operation.ts	Scoped authorization, one connection, transaction and RLS context.
apps/web/src/main.tsx	React root and shared base styles.
apps/web/src/host/	Composition, profile-aware shell, navigation and Start.
apps/web/src/features/	Browser domain/application contracts, HTTP gateways and React views.
apps/web/src/design/	Shared controls, identity tokens, surfaces and typography.
infra/database/	Role setup, ordered SQL migrations, CLI and PostgreSQL tests.
scripts/local/	Docker orchestration and explicit operator/demo tools.
config/ and .local-platform/	Tracked examples/source presets versus private live installation settings.
docs/planning/	Backlog, permanent items, active plans and completed evidence.

Domain code owns values and invariants. Application code owns use cases and ports. Adapters handle React, HTTP, SQL and source formats. Host composition connects them. Keep customer labels and source schemas in configuration/integration adapters.

Build output in dist/ is disposable. Database state is in the Docker volume. Empty/future package or host placeholders are not delivered features.

06 / Backend ownership map

Paths below are relative to apps/api/src/modules/. Domain/application code owns rules, use cases and ports; adapters own SQL, files and external protocols.

Directory	Responsibility / first implementation to read
platform-core/	Site ownership and configured locations. adapters/configuration/locations.ts; adapters/postgres/site-ownership.ts.
users-rbac/	Current profile/grant decisions and account administration. domain/profiles.ts; application/administration.ts.
authentication/	Password verification, throttle, sessions and revocation. application/authentication.ts; adapters/postgres.ts.
integrations/	Import lifecycle/RAW, source decoding and provider readiness. application/import-workflow.ts; application/provider-contracts.ts.
oip/	Data Analysis policy, exact measures and reports. application/reporting.ts; application/data-explorer.ts; adapters/postgres/.
shift-handover/	Journal rules and formal shift transfers. application/handover.ts; application/formal-records.ts; domain/handover.ts.
workforce/	Availability, teams, shifts, assignments and retained revisions. application/workforce.ts; adapters/postgres/store.ts.
maintenance/	Work lifecycle, permissions and explicit repair review/completion. application/maintenance.ts; adapters/postgres/store.ts.
assets/	Registered identity, composition/controllers, mapping review and source timeline. application/assets.ts; application/enrichment.ts.
improvement-tracking/	Owned actions, immutable evidence and source comparisons. application/improvement.ts.
asset-locator/	Private references, scenes, placement, revisions and source problem ports. application/locator.ts; adapters/postgres/store.ts.
operational-overview/	Source-authorized search and area composition. application/overview.ts; owner-defined source ports.
jobs/	Scoped durable leases/retry, idempotent effects and explicit handlers. application/jobs.ts; adapters/postgres/.
audit/	Safe material event append and separate inspection/retention. index.ts; adapters/postgres/append.ts.

apps/api/src/host/runtime.ts wires these owners. host/*-controller.ts translates HTTP and host/*-contracts.ts defines DTOs. The reviewed generated HTTP contract is apps/api/contracts/openapi.json. Start here when an endpoint or dependency is unclear.

persistence/site-operation.ts pins one connection, rechecks current grants and sets transaction-local organization/site/actor context. Forced RLS adds isolation; it does not replace business checks. Runtime uses iop_runtime, without owner/DDL privileges. Source-owned cross-module reads preserve independent authorization and history.

07 / Frontend layout and change map

apps/web/src/host/AnalyticalApp.tsx composes the whole application despite its historical name. WorkspaceApp.tsx owns session context, profile presentation, navigation and workspace selection. Shared shell/account/search controls surround the feature views; profile preview never changes the server actor.

Feature below apps/web/src/features/	Visible views / components to locate
analysis/	StartOverview, Workspace reports/filters/charts, ImportWorkspace, SourceFiles and preparation/profile editors. adapters/echarts/charts.ts renders server measures.
access/	LoginPanel and UserAdministration account/profile dialogs.
shift-handover/	HandoverWorkspace, MeetingCanvas, EntryForm/EntryDetail, SuccessReferences and InformationNotices. Formal transfer views live in the same feature.
workforce/	WorkforceWorkspace, PlanBoard/DailyPlan, WeeklySchedule, AssignmentForm and RecordDetails.
maintenance/	MaintenanceWorkspace, MaintenanceForm, RepairScope and MaintenanceStatusDialog.
assets/	AssetsWorkspace, AssetForm, AssetDetails, AssetAliasFields and AssetTimeline for the registry and source history.
assets-domain/	AssetDomainWorkspace, TypeCatalogEditor, EnrichmentEditor, MappingReview and CanonicalPareto for model/mapping management.
improvement-tracking/	ImprovementWorkspace, action details, evidence and retained comparisons.
asset-locator/	LocatorWorkspace, PlantOverview, SceneViewport and reusable Three.js geometry/renderer. Private source images are authenticated API resources, never public assets.
operational-overview/	Area context and global source-authorized search views.
integrations/	ConnectionsWorkspace plus application/connections.ts and the HTTP gateway for provider configuration/readiness.

Inside each feature, domain/ contains browser values; application/ has framework-free use cases; adapters/http/gateway.ts calls the API; adapters/react/ owns views and layout. Locator's adapters/three/ can be replaced without changing scene rules. Read models are not server persistence entities.

Shared path	Use
apps/web/src/design/components/	Reusable fields, compact filters, dates, dialogs, actions and navigation; index.ts exports the public controls.
apps/web/src/design/identity.ts and base.css	Shared semantic light/dark tokens, typography and base surfaces.
apps/web/src/localization/	English source/default text and German resources.
apps/web/src/contracts/schema.d.ts	Generated OpenAPI bindings; regenerate rather than edit.

apps/web/vite.config.ts fixes loopback ports and the API proxy. Feature CSS arranges views; use shared tokens/controls and docs/design/visual-identity.md for UI changes.

08 / Prepare once, before the trip

This recipe assumes the same Mac that already holds .local-platform/ and the database volume. A Git clone on another laptop does not contain the database or credentials. Moving to another machine needs a separately verified database transfer.

Open Terminal in the repository root:

```
cd ~/Desktop/VictorTilve/Switzerland/Galaxus/Techniker/IOP/iop-platform
nvm install
nvm use
node --version
npm --version
npm ci
```

Command	What it does
cd	Sets the root for every relative path in this guide. If the folder moved, use its actual path.
nvm install / nvm use	Installs/selects the .nvmrc Node version, currently 24.21.0. Run nvm use in each terminal.
node --version / npm --version	Confirm Node 24.21.0 and npm 10.9.2. The project requires Node 24, not Node 20.
npm ci	Installs exactly package-lock.json into the workspaces. It needs network on first preparation and replaces node_modules.

If nvm is installed but unavailable in a new terminal, load its shell initialization:

```
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && . "$NVM_DIR/nvm.sh"
nvm use
```

If npm differs from the documented version, prepare it while online:

```
npm install --global npm@10.9.2
```

Keep locally: the repository and node_modules, Docker Desktop, the existing postgres:17.6-bookworm image and persistent volume, and the complete private .local-platform/ installation. Preserve scope.json, users.json, mappings.json, handover.json, runtime.env, setup.env and the complete locator/ reference tree. Do not commit or place credentials in Vite.

What remains unchanged: organization/site/source IDs, source mappings, locations, existing password hashes/accounts and business data. The preparation on the next page copies browser-origin settings and reuses the runtime credential; it does not seed users. Never use demo:reset/demo:recreate, volume deletion or fixture --apply as startup steps.

09 / Prepare private native settings

Run this once from the root. Re-run if the repository moves or the runtime credential or original identity configuration changes. It writes only ignored private files. The added origin is the Vite browser address, not a permission grant.

```
node <<'NODE'
const fs = require('node:fs');
const path = require('node:path');
const dir = path.resolve('.local-platform/native');
fs.mkdirSync(dir, {recursive: true, mode: 0o700});
const users = JSON.parse(
  fs.readFileSync('.local-platform/config/users.json', 'utf8'));
users.origins = [...new Set([...users.origins,
  'http://127.0.0.1:5173'])];
if (users.origins.length > 4) throw Error('Review origin capacity.');
fs.writeFileSync(path.join(dir, 'users.json'),
  JSON.stringify(users, null, 2), {mode: 0o600});
const secret = fs.readFileSync('.local-platform/runtime.env', 'utf8')
  .split(/\r?\n/).find(x => x.startsWith('IOP_RUNTIME_PASSWORD='));
if (!secret) throw Error('Existing runtime credential is missing.');
const files = {
  IOP_CONFIG_FILE: '.local-platform/config/scope.json',
  IOP_LOCAL_IDENTITY_FILE: '.local-platform/native/users.json',
  IOP_MAPPING_CONFIG_FILE: '.local-platform/config/mappings.json',
  IOP_HANDOVER_CONFIG_FILE: '.local-platform/config/handover.json',
  IOP_LOCATOR_REFERENCE_ROOT: '.local-platform/locator'
};
const values = [
  'NODE_ENV=development', 'IOP_EXECUTION_MODE=local-demo',
  'IOP_AUTHENTICATION=password', 'IOP_TRANSPORT=native',
  'HOST=127.0.0.1', 'PORT=3000', 'IOP_DATABASE_MODE=local',
  'IOP_DATABASE_HOST=127.0.0.1', 'IOP_DATABASE_PORT=5442',
  'IOP_DATABASE_NAME=iop_local', secret,
  ...Object.entries(files).map(([key, file]) =>
    key + '=' + JSON.stringify(path.resolve(file)))
];
fs.writeFileSync(path.join(dir, 'runtime.env'),
  values.join('\n') + '\n', {mode: 0o600});
fs.writeFileSync(path.join(dir, 'compose.yaml'),
  'services:\n  database:\n    ports:\n' +
  '      - "127.0.0.1:5442:5432"\n', {mode: 0o600});
fs.chmodSync(dir, 0o700);
for (const file of ['users.json', 'runtime.env', 'compose.yaml'])
  fs.chmodSync(path.join(dir, file), 0o600);
console.log('Private native settings prepared.');
NODE
```

runtime.env contains the current runtime password plus explicit native settings. Node loads it only when --env-file is supplied. JSON file paths are absolute. The private users.json preserves configured user IDs and adds the allowed Vite origin. The private Compose overlay publishes container port 5432 on host loopback port 5442. It keeps project iop-platform-local and its platform-data volume. Locator reads the existing private host tree; it does not regenerate or publicly serve the PNGs.

The mode name local-demo is a historical execution label. With IOP_AUTHENTICATION=password it uses existing local login, not demo-user impersonation. Existing exported environment variables override env-file values; use a clean terminal if you previously exported incompatible IOP settings.

10 / Daily startup: database plus three terminals

Start Docker Desktop. Use compose.platform.yaml, not the separate foundation compose.yaml. From the root, stop the container applications before exposing the database. The single-host lease rejects a second API on this database.

```
docker compose -f compose.platform.yaml stop web api
docker compose -f compose.platform.yaml \
  -f .local-platform/native/compose.yaml up -d --wait database
npm run build --workspace @iop/api
```

stop web api leaves PostgreSQL and its volume intact. The overlay may recreate the database container to add the port; its persistent volume is reused. --wait checks container health. The API build creates current JavaScript once; it cleans API dist/. Do not pass a backup argument or run provisioning/seeding for this existing database.

Terminal A - compiler. Select Node with nvm use, then leave this running:

```
node node_modules/typescript/bin/tsc \
  -p apps/api/tsconfig.build.json --watch --preserveWatchOutput
```

TypeScript recompiles changed API source into apps/api/dist/. Wait for zero errors.

Terminal B - API. From the root with nvm use:

```
node --env-file=.local-platform/native/runtime.env \
  --watch --watch-preserve-output apps/api/dist/main.js
```

Node runs the compiled API and restarts when loaded JavaScript changes. The browser sees current backend behavior on its next request. Reload the page or repeat the action after an API restart; backend watch does not itself refresh React.

Terminal C - frontend. From the root with nvm use:

```
npm run dev:web
```

Vite serves source and sends React/CSS updates to the browser. Open http://127.0.0.1:5173 and log in with an existing account. Use 127.0.0.1 exactly, not localhost; the origin allowlist is exact. /api and /health proxy to port 3000.

Verify without changing data:

```
curl --fail http://127.0.0.1:5173/health
curl --fail http://127.0.0.1:5173/api/v1/session/context
```

Health should return status ok. Context should be enabled with password authentication; user null is normal before login. An enabled context is more informative than health.

11 / Follow a change through the code

Frontend edits to React/CSS use Vite hot updates. Changes to application state or composition may cause a full refresh or require reopening the screen. Unsaved forms can be lost on refresh. Existing API mutations write to the real retained database.

Backend edits follow source -> TypeScript output -> Node restart -> next browser request. Do not edit dist/ to implement a feature. If TypeScript reports an error, fix it before evaluating the browser. Use the three terminal logs and browser DevTools.

Example: a new Handover entry

1. EntryForm.tsx renders controls and collects the draft. application/entry-draft.ts handles category/condition draft transitions.

2. The browser application/handover.ts coordinates the use case; adapters/http/gateway.ts sends a same-origin /api/v1/handover request.

3. API host/handover-controller.ts receives HTTP input; handover-contracts.ts owns transport DTOs. host/runtime.ts supplies the configured catalog and actor.

4. API modules/shift-handover/application/handover.ts orchestrates current permissions, revision checks and writes. domain/handover.ts validates the content and classification.

5. adapters/postgres/store.ts uses persistence/site-operation.ts. Handover stores the current projection and immutable revisions on one authorized connection.

6. The response updates browser state and the view. Success may additionally invoke the Maintenance-owned completion contract within the same transaction.

Paths for steps 1-2 begin apps/web/src/features/shift-handover/. Paths for steps 3-5 begin apps/api/src/.

First reading exercise: inspect EntryForm.tsx, the HTTP gateway, handover-controller.ts and Handover.create, then follow validContent into the domain. Compare apps/api/test/handover-workflows.spec.ts with the implementation. Observe that Safety validation exists on the server as well as in the form.

Small UI exercise: locate a source/default label or layout in the owning React component; change it on your planned story branch and observe the hot update. Use shared design tokens. Add German text only in localization resources.

Small API exercise: change one authorized validation rule on a planned story, update a behavior test, watch compilation/restart, then repeat the browser request. Do not weaken permissions or RLS to simplify debugging.

In DevTools Network inspect URL, request fields, response status and Problem Details. A 400/403/409 usually reflects validation/permission/revision handling; a 5xx traceId correlates the sanitized server error. Generated bindings must match the API artifact.

12 / Checks, contracts and schema commands

Stop compiler/API watchers before commands that rebuild dist/. Root npm test, build, openapi, database tests and browser tests rebuild output; running them beside the watcher can trigger restarts or missing-module/port conflicts. Ctrl-C stops a terminal.

Root command	What it does
npm run typecheck	Checks API, web and database tooling TypeScript without emitting output.
npm test	Runs secret-tooling tests, builds, design/contract checks, API/web unit tests and database configuration tests. It does not test the live operator database.
npm test --workspace @iop/api	Runs API tests only; includes HTTP listener tests.
npm test --workspace @iop/web	Runs browser application/component tests without a real browser.
npm run test:database	Builds and runs isolated real PostgreSQL integration tests. Requires Docker; disposable Testcontainers do not use the operator volume.
npm run test:e2e	Builds and runs Chromium against temporary API/web servers. Stop native API first: the suite needs port 3000 and preview port 4173.
npm run check:design --workspace @iop/web	Checks shared visual-identity/boundary rules.
npm run check:secrets	Inspects indexed files for bounded secret/private-file patterns; manual review is still needed.

After an authorized HTTP contract change:

```
npm run openapi
npm run contract --workspace @iop/web
npm run contract:check --workspace @iop/web
```

openapi builds and exports Nest metadata into apps/api/contracts/openapi.json. contract regenerates apps/web/src/contracts/schema.d.ts. contract:check rejects drift. Review both generated diffs. There is no served Swagger UI.

Schema work only when your planned story adds reviewed migrations:

```
npm run db:build
node --env-file=.local-platform/native/runtime.env \
  --env-file=.local-platform/setup.env \
  infra/database/dist/cli.js migrate
```

db:build compiles the database CLI. migrate uses iop_migrator and ordered SQL/history; it is a real change to the existing database. The second private env file supplies the migration credential to this one-shot tool, not the running API. The original IOP-199 increment required no migration; later modules do. Check your current source and migration history. Do not run db:provision or seed commands merely to start native apps. Applied migrations stay immutable; append a new reviewed file rather than editing history.

13 / Troubleshooting and returning to Docker

Symptom	Check and response
Only health works / context disabled	Business activation is missing. Use the explicit native env file and current scope/mapping/identity files. Health alone does not prove access to data.
Another application host is active	Stop Docker api and any prior native API. One database permits one application host. Do not remove the lease.
Connection refused / startup failure	Start Docker Desktop; run the private database overlay; verify the 5442 loopback mapping and existing runtime credential. Do not generate new passwords.
403 from browser	Use http://127.0.0.1:5173 exactly; ensure that origin is in native/users.json; restart API after config changes. Current server grants can also deny the action.
Login required / expired session	Log in with an existing account. Session expiry or password/profile changes can invalidate access. View as does not change the server actor.
EADDRINUSE or Vite strict-port failure	Another process owns 3000/5173. Stop it; changing ports also requires proxy/origin settings.
Backend change seems invisible	Check compiler errors and Node restart log; repeat the request or refresh the page. Check that the browser is on 5173 rather than Docker 8080.
Config/mapping/location change invisible	Restart Node explicitly; startup reads JSON configuration once. Native users.json is a separate copy of the origin configuration.
Locator guide/reference is unavailable	Check IOP_LOCATOR_REFERENCE_ROOT and the existing private host tree. In IOP-211, normal navigation intentionally hides guides; Administrator enters Edit model to show them.
Maintenance Success reports conflict	Review current work revision and complete include/exclude repair scope. A failed Success closes nothing.
Administrator cannot publish Information	The default publisher profile is Team Leader, with current coordinator grants. Profile preview does not confer that identity.

End native development: press Ctrl-C in terminals A, B and C. The database may remain running for your next session. To stop it without deleting its volume:

```
docker compose -f compose.platform.yaml \
  -f .local-platform/native/compose.yaml stop database
```

Return to the same installed Docker images: stop all native terminals, then:

```
docker compose -f compose.platform.yaml up -d --wait --no-build database api web
docker compose -f compose.platform.yaml ps
```

Without the overlay or a build, this restores the standard services and removes the published database port, while reusing the volume and existing application images. Open http://127.0.0.1:8080. This was used for guide validation to retain installed IOP-211 while the PDF branch stays based on develop.

Deliberately rebuild from your current source: npm run local:up followed by npm run local:status also provisions/verifies roles, applies pending migrations, reconciles retained seed history and refreshes private references. It is not a schema/data rollback. Choose the correct source branch first. Keep the native files; never use compose down --volumes to switch execution modes.

14 / Operating infrastructure and data safety

Capability	What exists and how to use it
Local containers	Separate web/API/database with persistent data and private Locator references. Only web 8080 is exposed normally. The native overlay temporarily publishes PostgreSQL on loopback 5442; it does not copy the database.
Durable jobs	Scoped queued jobs retain attempts, leases and idempotent outcomes. Administrator initiation and current business grants are rechecked. A finite local tick can be run explicitly; only a diagnostic handler ships. Missing business/provider handlers remain pending, not delivered external work.
Full backup	Operator tooling captures a consistent database snapshot, private configuration and explicitly included references in an authenticated encrypted archive. Keep the separate random encryption key outside captured roots. Stop API/web and other application workers while keeping the database active for the maintenance lease.
Isolated restore verification	Restore into a new network-isolated database and private directory. Compare table data, ownership/grants and RLS metadata. Verification never replaces the active volume; a restore archive is not a startup seed.
Site material Audit	Explicit owners append safe events atomically. Inspection/retention use separate trusted operator credentials, not ordinary product-role browsing. Organization RBAC capture and platform security collection are still separate pending coverage.
CI and measurement	Repository CI definitions and local measurement tools exist. Hosted CI success, representative workloads, agreed thresholds and release acceptance require separate evidence. A local passing test is not a production-readiness claim.

Use docs/development/local-operations.md for exact backup/restore/job commands and docs/development/site-audit-integration.md for capture coverage. This guide does not run those mutating operator commands merely to start development.

Shared database means shared effects. Creating an entry, saving an asset or applying a migration from native mode changes what Docker will see next. Keep a verified backup before planned data/schema experiments. Synthetic automated tests use separate disposable targets; do not point fixtures at this retained installation.

A Git checkout changes code only. It does not undo database rows, migrations, private images or saved model revisions. Restore and deployment need their own reviewed operating procedure. No external plant control or hosted deployment is part of this local workflow.

15 / Delivery evidence and remaining inputs

The following are historical code-delivery gates, not test runs performed by this PDF refresh. The refresh separately validates its native recipe and PDF rendering.

Code snapshot	Recorded evidence
Published develop / IOP-203	API 509, web 192, PostgreSQL 319 and browser 57 checks passed. Local formal Handover, Tracking, Locator, model management, overview/search and operating infrastructure are delivered.
Local IOP-211 review	API 529, PostgreSQL 327 plus seven final Locator-capacity checks; geometry preparation and access refinements verified. Its visual follow-up passed web 231, browser 61 and 30 installed Locator checks. It is not yet merged/published.
Earlier IOP-200 recipe	Native compiler/watch/proxy and same-database operation were verified on 2026-10-09. This update retests the recipe with the current reviewed source and explicit Locator root; see its completed execution record for exact evidence.

2026-10-15 recipe verification: the reviewed native API and Vite reused the same database/volume. Compiler restart, React hot updates, authenticated module reads and a private Locator image passed. All 62 tables retained identical counts/content fingerprints; the original Docker images returned healthy with no published database port.

IOP-211 prepares 71 source scenes with 10,678 provisional components: 5,532 motor symbols, 2,817 photoelectric candidates, 2,324 conveyors/curves and five retained scanner references. The separate synthetic example remains. Counts describe model/evidence records, not surveyed equipment, live status or accepted identities.

Remaining input / scope	What is needed to finish
Physical plant registration	Measured common origin/orientation, scale, elevations and surveyed sensor positions. Separate Bereich drawings are not an aligned floorplan.
Canonical equipment links	Administrator reviews actual identities and source aliases, then explicitly links components; unknown candidates remain unverified.
External connections	Approved WinCC/Ultimo/Entra schema/version, endpoint, access and credentials; real transport implementation and contract verification.
Operational records	Real shifts and incoming managers for transfers; real action owners/dates/targets/exposure for meaningful improvement comparison. No operational data is invented.
Jobs and Audit	Real business/provider handlers; organization RBAC capture and separate platform-security authority/collection/retention.
Performance and release	Representative workloads, accepted thresholds and consolidated pilot measurements; hosted/shared-use controls and release decision.

See docs/development/platform-completion.md, the IOP-211 item/review branch and the canonical backlog. Local implementation, Git publication, stage/master promotion and shared deployment are distinct states. The optional older IOP-173 fixture writer is not a startup command and may not match current category rules.

16 / Offline routine and review

Before leaving: run the native startup once while online, install dependencies, open the required screens and verify login. Cache the Docker database image and Playwright browser if you want E2E tests. Keep this PDF locally; keep private settings and the existing volume on the same Mac. Git/network dependency downloads are not available offline, while the prepared native apps plus local database can run.

```
npx --no-install playwright install chromium
git status --short --branch
```

The first command downloads the browser when needed, so do it before travel. git status distinguishes your branch, tracked edits and untracked files. Do not assume copying source alone carries database state.

Start a future story after selecting its scope: inspect backlog/item/accepted ADRs, create its execution plan, then branch from develop. These are commands you run for your own review workflow; the guide does not authorize automatic merges/publication.

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/IOP-NNN-short-title
git diff
git diff --check
```

Use a real allocated story ID/name, and commit only that story. The fast-forward-only pull refuses divergent history. While offline, use your already synchronized develop. Run relevant checks; then create logical English commits. Preserve other review branches. Promotion remains story -> develop -> stage -> master with explicit owner authorization.

17 / Canonical sources

Use these maintained contracts when a feature, permission or command changes. Names following a directory prefix in the same cell share that prefix.

Source in the repository	Use
README.md, ARCHITECTURE.md	Entry point, architecture and source ownership.
docs/architecture/modules.md and data-model.md	Logical modules, contracts and persisted concepts.
docs/architecture/adr/	Read Accepted decisions relevant to your change; Proposed is not Accepted.
docs/planning/backlog.md and poc-delivery.md	Current story statuses and delivered evidence; older review dates are historical snapshots.
docs/development/platform-completion.md	Delivered modules, operating contracts and explicit remaining inputs.
docs/product/shift-handover.md; docs/development/formal-handover.md	Journal/Success versus exact shift transfer/closure/acknowledgement.
docs/product/workforce.md; maintenance-assets.md; asset-domain.md	Planning, technical work, registered identity and model ownership.
docs/development/asset-locator.md; improvement-tracking.md; operational-overview.md	Provisional models, action evidence and source-authorized search.
docs/product/integration-readiness.md; docs/development/local-operations.md	External adapter limits, jobs, backup and operator contracts.
docs/development/running-poc.md	Standard Docker operator workflow and seed provenance.
docs/development/local-configuration.md	Scope, origins and explicit environment/configuration rules.
infra/database/README.md	Roles, migrations, RLS and native database-tool contracts.
docs/development/testing-poc.md	Test layers and prerequisites.
docs/design/visual-identity.md	Shared UI roles, tokens and rendering checks.
docs/planning/workflow.md	Planning, English content, logical commits and publication convention.